UADE

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Programación Orientada a Objetos

Propuesta C: Sistema de Movilidad Compartida

Etapa 1: Modelado y aplicación de patrones

Docente	Maria Angela Leon
Integrantes	Astudillo Solis, Valentina (1211931) Pacheco, Maitena Alumine (1216103) Romero Chacon, Marcos Ignacio (1219650)
Fechas de entrega	**Etapa 1:** 25 de octubre de 2026 **Etapa 2:** 4 de noviembre de 2026

Github Repositorio : https://github.com/Mromeroch/POO_TPO.git

Índice

Consigna del trabajo3

1. Análisis del enunciado4

1.1 Actores4
1.2 Requerimientos funcionales (RF)4
1.3 Reglas de negocio (RN)5
1.4 Supuestos adoptados6

2. Diagrama de casos de uso9

2.1 Resumen de casos de uso9
2.2 Descripción textual de los casos de uso11

3. Diagrama de clases inicial22

3.1 Clases y atributos22
3.2 Relaciones y multiplicidades23

4. Diagramas de secuencia24

4.1 CU-05 · Retirar vehículo25
4.2 CU-08 · Devolver vehículo26

5. Aplicación de patrones de diseño27

5.1. Principios SOLID27
5.2. Patrones GRASP29
5.3. Síntesis de las decisiones de diseño32
5.4. Resultado esperado32

6. Diagrama de clases final33

6.1. Descripción general33
6.2. Clases y responsabilidades principales33
6.3. Relaciones principales del diagrama34
6.4. Nota de cambios respecto del diagrama de clases inicial35
6.5. Reglas de negocio reflejadas en el diseño final36

Consigna del trabajo

Resumen de lo que pide el trabajo práctico para esta etapa. Cada punto se desarrolla en una sección de este documento.

1	**Análisis del enunciado** Listar los actores, los requerimientos funcionales y las reglas de negocio. Si algo del enunciado es ambiguo, el grupo debe registrar el supuesto que adopta.
2	**Diagrama de casos de uso** Representar los actores y los casos de uso del sistema, con las relaciones de inclusión o extensión que correspondan. Incluir una breve descripción textual de cada caso de uso (actor, precondición, flujo principal, flujos alternativos).
3	**Diagrama de clases inicial** Identificar clases, atributos, métodos, visibilidad, multiplicidades y relaciones (asociación, agregación, composición, herencia). Debe incluir al menos una jerarquía de herencia con sobreescritura de métodos.
4	**Diagramas de secuencia** Elegir dos casos de uso del diagrama anterior (uno de ellos con un flujo alternativo o de error) y desarrollar para cada uno su diagrama de secuencia, con objetos, mensajes, retornos y fragmentos alt, opt o loop cuando corresponda.
5	**Aplicación de patrones** Revisar el diseño y aplicar **3 principios SOLID** a elección (por ejemplo, responsabilidad única, abierto/cerrado, sustitución de Liskov, segregación de interfaces, inversión de dependencias) y **4 patrones GRASP** a elección (por ejemplo, Experto, Creador, Controlador, Bajo acoplamiento, Alta cohesión, Polimorfismo, Fabricación pura, Indirección, Variaciones protegidas). Para cada uno se debe documentar: **qué problema del diseño detectó el grupo, qué decisión tomó, en qué clases se ve reflejado y qué ventaja aporta**. No alcanza con nombrar el patrón: hay que justificarlo.
6	**Diagrama de clases final** Rediseñar el diagrama de clases incorporando los cambios derivados de los pasos anteriores. Se debe adjuntar una nota que indique qué cambió respecto de la versión inicial y por qué.

1. Análisis del enunciado

En esta sección se identifican los actores, los requerimientos funcionales, las reglas de negocio y los supuestos que adopta el grupo ante lo que el enunciado no define.

1.1 Actores

Actor	Rol en el sistema
Usuario	Persona registrada que alquila y utiliza los vehículos de movilidad compartida (bicicleta común, bicicleta y monopatín eléctricos).
Administrador	Encargado de emitir los reportes de viajes por usuario, uso por estación y vehículos con más fallas.
Operador del Sistema	Encargado de redistribuir vehículos entre estaciones, atender las alertas de ocupación, generar y asignar órdenes de mantenimiento, reportar fallas y corregir viajes cerrados mediante notas de ajuste.
Técnico de Mantenimiento	Encargado de atender las órdenes de mantenimiento y cerrarlas registrando tareas y repuestos.
Sistema de Pago Externo	Entidad externa que procesa las recargas de saldo.

1.2 Requerimientos funcionales (RF)

ID	Requerimiento
RF01	El sistema debe permitir **registrar usuarios** (nombre, documento, correo y fecha de alta) y asociarles una cuenta virtual para gestionar su saldo.
RF02	El sistema debe permitir al usuario **asociar medios de pago** (tarjeta o billetera virtual) a la cuenta, recargar saldo y registrar cada movimiento (recarga o débito) con fecha e importe.
RF03	El sistema debe permitir al usuario **suscribirse** a un plan mensual.
RF04	El sistema debe permitir al usuario **retirar un vehículo** disponible en una estación, registrando el inicio del viaje.
RF05	El sistema debe permitir al usuario **cancelar** un retiro sin cargo.
RF06	El sistema debe permitir al usuario **devolver** el vehículo en una estación, registrando el fin del viaje, la distancia y los puntos del recorrido.
RF07	El sistema debe **calcular** el importe del viaje y debitarlo de la cuenta del usuario.
RF08	El sistema debe **gestionar el estado** de cada vehículo (disponible, en uso, en reparación, batería baja o fuera de servicio).
RF09	El sistema debe **detectar** los viajes que no se devolvieron y reportar el vehículo como perdido.
RF10	El sistema debe permitir a usuarios y operadores **reportar** fallas de un vehículo.
RF11	El sistema debe permitir al operador **generar órdenes** de mantenimiento asignadas a un técnico, y al técnico cerrarlas.
RF12	El sistema debe **calcular la ocupación** de cada estación y alertar al operador de los desbalances.
RF13	El sistema debe permitir al operador **registrar traslados** de vehículos entre estaciones.
RF14	El sistema debe permitir al operador **corregir un viaje cerrado** mediante una nota de ajuste.

ID	Requerimiento
RF15	El sistema debe permitir al administrador **emitir los reportes** de viajes por usuario, uso por estación y vehículos con más fallas.

1.3 Reglas de negocio (RN)

ID	Regla
Usuarios y saldo	
RN01	Un usuario solo puede retirar un vehículo si su saldo alcanza el mínimo establecido, está habilitado y no tiene otro viaje abierto.
RN02	Si al terminar un viaje el saldo queda negativo, el usuario queda inhabilitado hasta regularizarlo.
Vehículos	
RN03	Solo un vehículo disponible puede retirarse. Un eléctrico con batería menor al 15% pasa a batería baja y no se ofrece.
RN04	Un vehículo con 3 reportes de falla en 30 días pasa a mantenimiento obligatorio.
RN05	Un vehículo con una orden de mantenimiento abierta permanece en reparación y no puede retirarse.
RN06	Una orden de mantenimiento no se cierra sin registrar el técnico, la fecha y las tareas realizadas.
Estaciones	
RN07	Cada estación tiene capacidad fija. No se puede devolver un vehículo en una estación llena ni fuera de una estación.
RN08	Con una estación llena, el usuario dispone de 10 minutos adicionales sin cargo para llegar a otra.
RN09	Una ocupación menor al 20% o mayor al 90% genera una alerta de redistribución.
Viajes y tarifas	
RN10	La tarifa es un cargo de desbloqueo más un precio por minuto según el tipo de vehículo.
RN11	Un retiro cancelado dentro de los 2 minutos no tiene cargo.
RN12	El tiempo máximo de un viaje es de 2 horas. Pasado ese límite, el minuto se cobra con un recargo del 50%.
RN13	A las 24 horas sin devolución el vehículo se reporta como perdido y se aplica un cargo fijo.
RN14	El primer viaje de cada usuario es gratis hasta 15 minutos, y el plan mensual incluye 20 viajes de hasta 30 minutos. No se acumulan: se aplica la más favorable para el usuario.
RN15	Un viaje cerrado no se modifica; el operador corrige con una nota de ajuste.

1.4 Supuestos adoptados

Decisiones del grupo sobre los puntos que el enunciado deja abiertos. Los valores numéricos son de referencia y pueden ajustarse si el supuesto queda documentado.

ID	Tema	Supuesto
S01	**Saldo mínimo de retiro**	El saldo mínimo para retirar un vehículo equivale al cargo de desbloqueo más 30 minutos de uso de la unidad seleccionada: $800 bicicleta común, $1.650 bicicleta eléctrica y $1.800 monopatín eléctrico.

ID	Tema	Supuesto
S02	**Cálculo de la duración**	La duración del viaje se mide con la fecha y hora registradas al retirar y al devolver el vehículo. Los minutos se redondean hacia arriba: una fracción cuenta como minuto completo.
S03	**Estación llena**	Los 10 minutos adicionales sin cargo se cuentan desde el primer intento fallido de devolución por estación llena, y el sistema registra ese instante. Al finalizar el viaje se descuentan del tiempo facturable hasta 10 minutos contados desde ese instante. Pasado ese plazo se vuelve a cobrar. Solo cuenta el primer intento.
S04	**Recargo por exceso de tiempo**	El recargo del 50% aplica solo a los minutos posteriores al minuto 120 del viaje.
S05	**Promociones**	Primer viaje gratis: se bonifica el desbloqueo y los primeros 15 minutos. Plan mensual: se bonifica el desbloqueo y hasta 30 minutos por viaje. En ambos casos el excedente se cobra a tarifa normal. El sistema calcula el costo sin promoción y con cada promoción aplicable, y cobra el menor. El viaje guarda el nombre de la promoción aplicada. El primer viaje gratis se otorga solo si el usuario no tiene viajes finalizados previos.
S06	**Plan mensual**	El plan dura 30 días corridos, su precio lo define el administrador y se debita de la cuenta al suscribirse. Cada usuario tiene como máximo un plan vigente. Cada viaje finalizado con plan vigente consume 1 de los 20 viajes incluidos, aunque supere los 30 minutos (el excedente se cobra a tarifa normal). Con los 20 viajes agotados, el plan deja de ser aplicable.
S07	**Vehículo perdido**	El cargo fijo por vehículo perdido es de $20.000. La detección se ejecuta al iniciar el sistema y cada vez que el operador la solicita. Al superar las 24 horas sin devolución, el viaje pasa a PERDIDO, se debita el cargo, el vehículo queda FUERA_DE_SERVICIO y, si el saldo queda negativo, el usuario se inhabilita.
S08	**Fallas y mantenimiento obligatorio**	Cada reporte de falla guarda fecha, descripción y quién lo hizo (documento o legajo). La ventana de 30 días se cuenta hacia atrás desde la fecha del nuevo reporte. Al tercer reporte en esa ventana, el sistema pasa el vehículo a EN_REPARACION y alerta al operador para que genere la orden y asigne un técnico. Si el vehículo está EN_USO, el cambio de estado se aplica al finalizar el viaje.
S09	**Alta de usuario y cuenta**	Al registrar un usuario, el sistema crea su cuenta automáticamente con saldo inicial $0. Sin una recarga previa, el usuario no alcanza el saldo mínimo para retirar un vehículo.
S10	**Pagos y débito del viaje**	El importe del viaje se descuenta del saldo al devolver el vehículo, sin intervención del usuario. El saldo solo aumenta con recargas que inicia el usuario: el sistema nunca cobra directamente a la tarjeta o billetera. El Sistema de Pago Externo se simula: aprueba recargas de importe mayor a $0 con un medio asociado, y una recarga rechazada no modifica el saldo. Un usuario puede tener varios medios de pago.
S11	**Estados del viaje**	Un viaje puede estar EN_CURSO, FINALIZADO, CANCELADO o PERDIDO. Solo los viajes FINALIZADOS cuentan como viajes realizados.
S12	**Cancelación de retiro**	El usuario puede cancelar dentro de los 2 minutos posteriores al retiro. El vehículo vuelve a la estación de retiro en estado DISPONIBLE y el viaje queda CANCELADO con costo $0. Pasado ese plazo debe devolverlo normalmente.

ID	Tema	Supuesto
S13	**Distancia del viaje**	La distancia es la suma de los tramos entre puntos consecutivos del recorrido, calculados con sus coordenadas. Si el viaje no tiene puntos, se usa la distancia informada al devolver. Al cerrar el viaje se suma a los kilómetros del vehículo.
S14	**Batería de eléctricos**	Al cerrar un viaje, los eléctricos descuentan batería en proporción a la distancia (porcentaje por km definido como constante por tipo). Si queda por debajo del 15% pasan a BATERIA_BAJA y no se ofrecen. La batería se repone al 100% con un traslado del operador o al cerrar una orden de mantenimiento.
S15	**Alertas de ocupación**	Ocupación = vehículos guardados / capacidad de la estación. Se evalúa después de cada retiro, devolución y traslado. La alerta es VACIA (menos del 20%) o SATURADA (más del 90%).
S16	**Tarifa vigente**	Se usa la tarifa del tipo de vehículo con la mayor fecha de vigencia menor o igual a la fecha de inicio del viaje. El viaje conserva la tarifa con la que se calculó.
S17	**Rehabilitación de usuarios**	Un usuario inhabilitado vuelve a estar habilitado cuando, después de una recarga o de una nota de ajuste, su saldo es mayor o igual a $0.
S18	**Órdenes de mantenimiento**	Las órdenes tienen estado ABIERTA o CERRADA, y un vehículo tiene como máximo una orden abierta. Al generarse la orden, el vehículo sale de la estación y queda EN_REPARACION. Al cerrarla, el técnico indica la estación (con lugar) donde el vehículo queda DISPONIBLE, con la batería repuesta.
S19	**Nota de ajuste**	Registra viaje, importe (positivo acredita al usuario, negativo debita), motivo, operador y fecha. Genera un movimiento de tipo AJUSTE en la cuenta y no modifica el viaje original. Un viaje puede tener varias notas.
S20	**Traslados de vehículos**	El operador mueve vehículos DISPONIBLES o con BATERIA_BAJA desde una estación origen hacia otra con lugar. No se trasladan vehículos EN_USO ni EN_REPARACION. Al trasladar, la batería se repone y BATERIA_BAJA pasa a DISPONIBLE.
S21	**Reloj del sistema**	La fecha y hora actuales se obtienen de un reloj inyectable, para poder probar los límites temporales (2 min, 10 min, 120 min, 24 h y 30 días).
S22	**Interfaz y roles**	La interfaz es por consola, con un menú separado por rol (usuario, operador, técnico y administrador). No hay autenticación.
S23	**Persistencia**	Los datos se guardan en archivos de texto (CSV o JSON) dentro de una carpeta data. Se cargan al iniciar y se guardan después de cada operación que modifica el estado.
S24	**Reportes**	Viajes por usuario: por documento y rango de fechas opcional, ordenados por fecha. Uso por estación: cantidad de retiros y devoluciones. Vehículos con más fallas: ranking descendente por cantidad total de reportes.
S25	**Moneda e importes**	Todos los importes están en pesos argentinos y se redondean a 2 decimales. Los valores de tarifa y el cargo por pérdida son de referencia y se guardan como constantes configurables.

2. Diagrama de casos de uso

El diagrama representa a los cuatro actores humanos del sistema y a sus casos de uso, con las relaciones de inclusión («include») y extensión («extend») que corresponden. Las notas S01 y S05 recuerdan los supuestos que condicionan el retiro y las promociones.

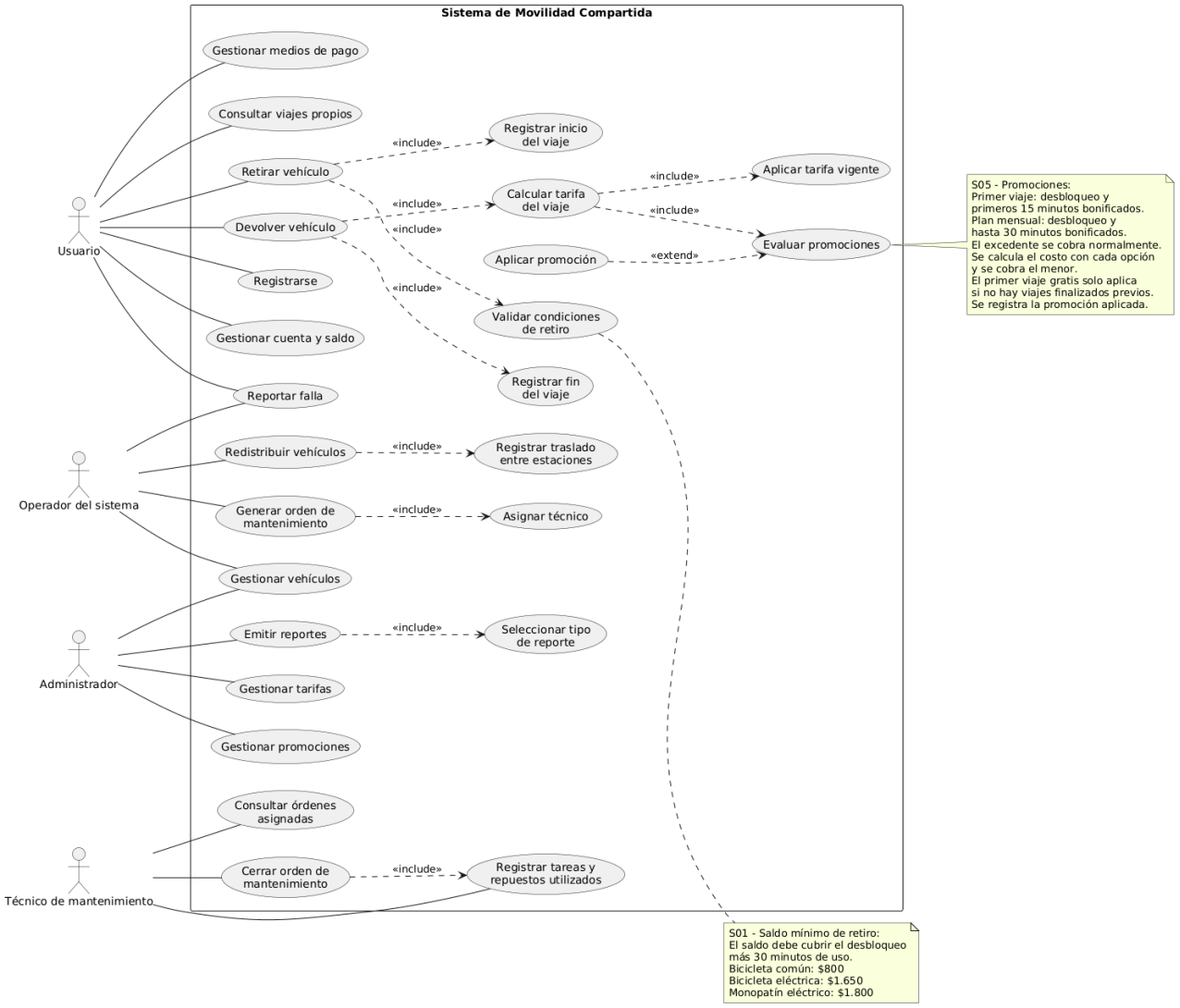


Figura 1. Diagrama de casos de uso del sistema de movilidad compartida.

2.1 Resumen de casos de uso

ID	Caso de uso	Actor principal	Relación
CU-01	Registrarse	Usuario	Caso de uso principal
CU-02	Gestionar cuenta y saldo	Usuario	Caso de uso principal
CU-03	Gestionar medios de pago	Usuario	Caso de uso principal
CU-04	Consultar viajes propios	Usuario	Caso de uso principal
CU-05	Retirar vehículo	Usuario	Caso de uso principal
CU-06	Validar condiciones de retiro	Usuario	«include» de CU-05
CU-07	Registrar inicio del viaje	Usuario	«include» de CU-05
CU-08	Devolver vehículo	Usuario	Caso de uso principal

ID	Caso de uso	Actor principal	Relación
CU-09	Registrar fin del viaje	Usuario	«include» de CU-08
CU-10	Calcular tarifa del viaje	Usuario	«include» de CU-08
CU-11	Aplicar tarifa vigente	Usuario	«include» de CU-10
CU-12	Evaluar promociones	Usuario	«include» de CU-10
CU-13	Aplicar promoción	Usuario	«extend» de CU-12
CU-14	Reportar falla	Usuario y operador del sistema	Caso de uso principal
CU-15	Gestionar vehículos	Operador del sistema y administrador	Caso de uso principal
CU-16	Generar orden de mantenimiento	Operador del sistema	Caso de uso principal
CU-17	Asignar técnico	Operador del sistema	«include» de CU-16
CU-18	Consultar órdenes asignadas	Técnico de mantenimiento	Caso de uso principal
CU-19	Registrar tareas y repuestos utilizados	Técnico de mantenimiento	«include» de CU-20
CU-20	Cerrar orden de mantenimiento	Técnico de mantenimiento	Caso de uso principal
CU-21	Redistribuir vehículos	Operador del sistema	Caso de uso principal
CU-22	Registrar traslado entre estaciones	Operador del sistema	«include» de CU-21
CU-23	Gestionar tarifas	Administrador	Caso de uso principal
CU-24	Gestionar promociones	Administrador	Caso de uso principal
CU-25	Emitir reportes	Administrador	Caso de uso principal
CU-26	Seleccionar tipo de reporte	Administrador	«include» de CU-25

2.2 Descripción textual de los casos de uso

Las siguientes fichas documentan los casos de uso del diagrama. Se especifican el actor, las precondiciones, el flujo principal y los flujos alternativos o de error, como exige la consigna de la Etapa 1.

CU-01	Registrarse
Actor principal	Usuario.
Precondición	La persona todavía no tiene una cuenta registrada en el sistema.
Flujo principal	**1.** El usuario selecciona la opción de registro. **2.** El sistema solicita nombre, documento y correo electrónico. **3.** El usuario ingresa sus datos. **4.** El sistema valida que los datos sean correctos y que no exista otro usuario con el mismo documento. **5.** El sistema registra al usuario y crea su cuenta de saldo. **6.** El sistema confirma el registro.
Flujos alternativos	**A1.** Si falta un dato obligatorio, el sistema solicita completarlo. **A2.** Si el documento ya está registrado, el sistema informa que el usuario ya existe y no crea una cuenta duplicada.

CU-02	Gestionar cuenta y saldo
Actor principal	Usuario.
Precondición	El usuario se encuentra registrado.
Flujo principal	**1.** El usuario accede a la sección de su cuenta. **2.** El sistema muestra el saldo disponible y los movimientos registrados. **3.** El usuario selecciona la opción de recargar saldo. **4.** El sistema solicita el importe y el medio de pago asociado. **5.** El sistema procesa la recarga. **6.** El sistema actualiza el saldo y registra el movimiento con su fecha e importe. **7.** El sistema confirma la operación.
Flujos alternativos	**A1.** Si el importe no es válido, se solicita ingresarlo nuevamente. **A2.** Si el medio de pago rechaza la operación, no se acredita el importe y se informa el resultado. **A3.** Si el usuario tiene saldo negativo, puede consultar su cuenta, pero debe regularizar la deuda para recuperar la habilitación para viajar.

CU-03	Gestionar medios de pago
Actor principal	Usuario.
Precondición	El usuario tiene una cuenta registrada.
Flujo principal	**1.** El usuario accede a la gestión de medios de pago. **2.** El sistema muestra los medios asociados. **3.** El usuario ingresa los datos de una tarjeta o billetera virtual. **4.** El sistema valida los datos necesarios. **5.** El sistema registra el medio de pago asociado a la cuenta. **6.** El sistema confirma la operación.

Flujos alternativos	**A1.** Si los datos son incorrectos, el sistema solicita corregirlos. **A2.** Si el medio de pago no puede asociarse, el sistema informa el error y conserva los medios previamente registrados.

CU-04	Consultar viajes propios
Actor principal	Usuario.
Precondición	El usuario se encuentra registrado.
Flujo principal	**1.** El usuario accede a su historial de viajes. **2.** El sistema busca los viajes asociados a su cuenta. **3.** El sistema muestra las fechas, los vehículos, las estaciones de retiro y devolución, la duración, la distancia y el importe correspondiente.
Flujos alternativos	**A1.** Si no existen viajes registrados, el sistema informa que el historial está vacío.

CU-05	Retirar vehículo
Actor principal	Usuario.
Precondición	El usuario tiene una cuenta registrada y existe un vehículo disponible en una estación.
Flujo principal	**1.** El usuario selecciona una estación y un vehículo disponible. **2.** El sistema ejecuta Validar condiciones de retiro (CU-06). **3.** Si las condiciones se cumplen, el sistema registra el inicio del viaje mediante Registrar inicio del viaje (CU-07). **4.** El sistema asocia el viaje con el usuario, el vehículo y la estación de retiro. **5.** El sistema cambia el estado del vehículo a «en uso». **6.** El sistema confirma el retiro.
Flujos alternativos	**A1.** Si el saldo no alcanza el mínimo requerido, el sistema rechaza el retiro. **A2.** Si el usuario tiene otro viaje abierto, no puede retirar otro vehículo. **A3.** Si el vehículo no está disponible o tiene batería insuficiente, el retiro se rechaza. **A4.** Si falla el registro del viaje, el sistema no debe dejar el vehículo marcado como retirado sin un viaje válido.

CU-06	Validar condiciones de retiro
Actor principal	Usuario, mediante el caso de uso «Retirar vehículo».
Precondición	El usuario seleccionó un vehículo y una estación.
Flujo principal	**1.** El sistema verifica que el usuario esté habilitado. **2.** Comprueba que no tenga otro viaje abierto. **3.** Comprueba que el saldo alcance el mínimo establecido. **4.** Verifica que el vehículo esté disponible. **5.** Si es eléctrico, verifica que su batería no sea inferior al 15 %. **6.** El sistema informa que las condiciones se cumplen.
Flujos alternativos	**A1.** Si falla alguna comprobación, el sistema informa el motivo y no autoriza el retiro. **A2.** Si el vehículo es una bicicleta común, no se realiza la comprobación de batería.

CU-07	Registrar inicio del viaje
Actor principal	Usuario, mediante el caso de uso «Retirar vehículo».
Precondición	Las condiciones de retiro fueron validadas correctamente.
Flujo principal	**1.**El sistema crea un registro de viaje. **2.**Registra el usuario, el vehículo, la estación de retiro y la fecha y hora de inicio. **3.**Establece el estado del viaje como «en curso». **4.**Vincula el vehículo al viaje. **5.**Confirma el inicio.
Flujos alternativos	**A1.**Si no se puede guardar el viaje, se informa el error y no se confirma el retiro.

CU-08	Devolver vehículo
Actor principal	Usuario.
Precondición	El usuario tiene un viaje abierto y el vehículo está asociado a ese viaje.
Flujo principal	**1.**El usuario selecciona la opción de finalizar el viaje e indica la estación de devolución. **2.**El sistema verifica que la estación sea válida y tenga capacidad disponible. **3.**El sistema ejecuta Registrar fin del viaje (CU-09). **4.**El sistema ejecuta Calcular tarifa del viaje (CU-10). **5.**El sistema registra el importe correspondiente y el débito en la cuenta del usuario. **6.**El sistema registra la estación de devolución, la hora de finalización y la distancia recorrida. **7.**El sistema actualiza el estado del viaje a «finalizado» y el del vehículo a «disponible». **8.**El sistema confirma la devolución.
Flujos alternativos	**A1.**Si la estación está llena, el sistema rechaza la devolución en ese punto y permite seleccionar otra estación. **A2.**Si el usuario encuentra la estación llena, se contempla la extensión de hasta 10 minutos adicionales sin cargo para llegar a otra estación, según la regla de negocio. **A3.**Si la devolución se intenta fuera de una estación, el sistema no la acepta como devolución válida. **A4.**Si el débito deja el saldo negativo, el sistema registra la deuda y el usuario queda inhabilitado hasta regularizarla. **A5.**Si el vehículo no se devuelve dentro de las 24 horas, se aplica el procedimiento de reporte como perdido y el cargo fijo definido para esa situación.

CU-09	Registrar fin del viaje
Actor principal	Usuario, mediante el caso de uso «Devolver vehículo».
Precondición	Existe un viaje en curso y se ha seleccionado una estación de devolución válida.
Flujo principal	**1.**El sistema registra la fecha y hora de finalización. **2.**Registra la estación de devolución. **3.**Consolida la distancia recorrida y los puntos del recorrido registrados durante el viaje. **4.**Actualiza el estado del viaje a «finalizado».
Flujos alternativos	**A1.**Si la estación está llena o no es válida, el sistema no finaliza el viaje en esa ubicación. **A2.**Si no existe un viaje abierto asociado al usuario, el sistema informa que no hay ningún viaje que finalizar.

CU-10	Calcular tarifa del viaje
Actor principal	Usuario, mediante el caso de uso «Devolver vehículo».
Precondición	El sistema dispone de los datos del viaje y de la tarifa aplicable.
Flujo principal	**1.** El sistema determina la duración del viaje. **2.** Identifica el tipo de vehículo utilizado. **3.** Ejecuta Aplicar tarifa vigente (CU-11). **4.** Ejecuta Evaluar promociones (CU-12). **5.** Calcula el importe final, contemplando los recargos que correspondan. **6.** Devuelve el importe al proceso de devolución.
Flujos alternativos	**A1.** Si el viaje supera las dos horas, se aplica el recargo del 50 % al tiempo que exceda ese límite, según la interpretación adoptada por el grupo. **A2.** Si corresponde una promoción, se utiliza el beneficio más favorable para el usuario, sin acumular promociones.

CU-11	Aplicar tarifa vigente
Actor principal	Usuario, mediante el cálculo de tarifa.
Precondición	Existe un viaje con tipo de vehículo y duración conocidos.
Flujo principal	**1.** El sistema identifica el tipo de vehículo. **2.** Busca la tarifa vigente para ese tipo y para la fecha del viaje. **3.** Calcula el cargo de desbloqueo más el importe por minuto utilizado. **4.** Devuelve el importe calculado.
Flujos alternativos	**A1.** Si no existe una tarifa vigente, el sistema informa el problema y no confirma un importe incorrecto. *Como referencia inicial, la propuesta establece las siguientes tarifas: bicicleta común, $200 de desbloqueo más $20 por minuto; bicicleta eléctrica, $300 más $45 por minuto; monopatín eléctrico, $300 más $50 por minuto.*

CU-12	Evaluar promociones
Actor principal	Usuario, mediante el cálculo de tarifa.
Precondición	Existe un viaje y se dispone de la información del usuario, de su plan mensual y de las promociones vigentes.
Flujo principal	**1.** El sistema consulta las promociones vigentes. **2.** Comprueba si el usuario puede acceder al beneficio del primer viaje gratuito de hasta 15 minutos. **3.** Comprueba si corresponde el beneficio del plan mensual de hasta 20 viajes de hasta 30 minutos. **4.** Compara los beneficios aplicables. **5.** Determina la promoción más favorable sin acumular beneficios. **6.** Devuelve el resultado al cálculo de tarifa.
Flujos alternativos	**A1.** Si ninguna promoción cumple sus condiciones, se mantiene la tarifa normal. **A2.** Si existen varias promociones aplicables, se utiliza la más favorable para el usuario.

CU-13	Aplicar promoción

Actor principal	Usuario, mediante el cálculo de tarifa.
Precondición	El sistema detectó una promoción cuyas condiciones se cumplen.
Flujo principal	**1.**El sistema determina el beneficio correspondiente. **2.**Aplica la reducción o gratuidad establecida. **3.**Registra el beneficio aplicado al viaje. **4.**Devuelve el importe resultante.
Flujos alternativos	**A1.**Si la promoción ya no está vigente o no cumple sus condiciones, no se aplica y se utiliza la siguiente opción válida o la tarifa normal.

CU-14	Reportar falla
Actor principal	Usuario y operador del sistema.
Precondición	El actor identifica un problema en un vehículo.
Flujo principal	**1.**El actor selecciona el vehículo afectado. **2.**Ingresa una descripción de la falla. **3.**El sistema registra el reporte con la fecha y el vehículo correspondiente. **4.**El sistema actualiza el historial de fallas del vehículo. **5.**El sistema comprueba si corresponde activar el mantenimiento obligatorio. **6.**El sistema confirma el registro del reporte.
Flujos alternativos	**A1.**Si el vehículo no existe, el sistema rechaza el reporte. **A2.**Si el vehículo acumula tres reportes de falla en 30 días, pasa a mantenimiento obligatorio. **A3.**Si el vehículo eléctrico tiene una batería inferior al 15 %, debe figurar como «batería baja» y no ofrecerse para nuevos retiros.

CU-15	Gestionar vehículos
Actor principal	Operador del sistema y administrador.
Precondición	El actor tiene acceso a las funciones de gestión de vehículos.
Flujo principal	**1.**El actor consulta el listado de vehículos. **2.**El sistema muestra el código, el tipo, la ubicación y el estado de cada vehículo. **3.**El actor selecciona un vehículo para consultar o gestionar su situación. **4.**El sistema muestra su información y los reportes de fallas asociados. **5.**Si corresponde, el operador inicia la generación de una orden de mantenimiento.
Flujos alternativos	**A1.**Si no existen vehículos registrados, el sistema informa que no hay datos disponibles. **A2.**Un vehículo en uso no puede ofrecerse para otro retiro. **A3.**Un vehículo fuera de servicio o en reparación no puede ofrecerse para su utilización.

CU-16	Generar orden de mantenimiento
Actor principal	Operador del sistema.
Precondición	Existe un vehículo con una falla que requiere intervención.

Flujo principal	**1.**El operador selecciona el vehículo afectado. **2.**El sistema consulta los reportes de falla. **3.**El operador registra la descripción del trabajo y la prioridad. **4.**El sistema solicita la asignación de un técnico mediante Asignar técnico (CU-17). **5.**El sistema registra la orden con su fecha de apertura. **6.**El sistema cambia el estado del vehículo a «en reparación», según corresponda. **7.**El sistema confirma la creación de la orden.
Flujos alternativos	**A1.**Si el vehículo no existe o no presenta una falla que justifique la orden, se informa el problema. **A2.**Si no hay técnicos disponibles para asignar, la orden no debe quedar cerrada ni asignada a un técnico inexistente.

CU-17	Asignar técnico
Actor principal	Operador del sistema.
Precondición	Existe una orden de mantenimiento pendiente de asignación.
Flujo principal	**1.**El operador consulta los técnicos registrados. **2.**Selecciona un técnico. **3.**El sistema vincula el técnico con la orden. **4.**El sistema confirma la asignación.
Flujos alternativos	**A1.**Si no hay técnicos disponibles, el sistema informa que la orden queda pendiente de asignación.

CU-18	Consultar órdenes asignadas
Actor principal	Técnico de mantenimiento.
Precondición	El técnico se encuentra registrado en el sistema.
Flujo principal	**1.**El técnico accede a sus órdenes. **2.**El sistema busca las órdenes asignadas a su legajo. **3.**Muestra los vehículos involucrados, las fechas, las prioridades y las descripciones de los trabajos. **4.**El técnico selecciona una orden para consultar sus detalles.
Flujos alternativos	**A1.**Si no tiene órdenes asignadas, el sistema informa que no existen órdenes pendientes.

CU-19	Registrar tareas y repuestos utilizados
Actor principal	Técnico de mantenimiento.
Precondición	El técnico tiene una orden asignada y puede registrar el trabajo realizado.
Flujo principal	**1.**El técnico selecciona la orden. **2.**Registra las tareas realizadas. **3.**Registra los repuestos utilizados, si corresponde. **4.**El sistema guarda la información asociada a la orden. **5.**El sistema confirma el registro.
Flujos alternativos	**A1.**Si la orden no existe o no está asignada al técnico, el sistema rechaza el registro. **A2.**Si los datos obligatorios están incompletos, el sistema solicita corregirlos.

CU-20	Cerrar orden de mantenimiento
Actor principal	Técnico de mantenimiento.
Precondición	La orden está asignada al técnico y el trabajo fue realizado.
Flujo principal	**1.**El técnico selecciona la orden. **2.**El sistema comprueba que existan tareas realizadas registradas. **3.**El técnico confirma el cierre. **4.**El sistema registra la fecha de cierre y el responsable. **5.**El sistema actualiza el estado de la orden. **6.**El sistema actualiza el estado del vehículo según el resultado del mantenimiento. **7.**El sistema confirma el cierre.
Flujos alternativos	**A1.**Si no se registraron las tareas realizadas o falta la fecha de cierre, el sistema impide cerrar la orden. **A2.**Si el vehículo continúa presentando una falla, permanece fuera de disponibilidad hasta que se encuentre en condiciones de uso.

CU-21	Redistribuir vehículos
Actor principal	Operador del sistema.
Precondición	Existe al menos una estación con vehículos que pueden trasladarse y otra estación que necesita recibirlos.
Flujo principal	**1.**El operador consulta la ocupación de las estaciones. **2.**El sistema identifica las estaciones con menos del 20 % o más del 90 % de ocupación. **3.**El operador selecciona la estación de origen, la de destino y los vehículos que se trasladarán. **4.**El sistema verifica que los vehículos puedan trasladarse y que la estación de destino tenga capacidad. **5.**El sistema ejecuta Registrar traslado entre estaciones (CU-22). **6.**El sistema actualiza la ubicación de los vehículos. **7.**El sistema confirma la redistribución.
Flujos alternativos	**A1.**Si la estación de destino no tiene capacidad, el sistema rechaza el traslado o solicita seleccionar otra estación. **A2.**Si un vehículo está en uso, en reparación o fuera de servicio, no se incluye entre los vehículos disponibles para redistribución.

CU-22	Registrar traslado entre estaciones
Actor principal	Operador del sistema.
Precondición	El traslado fue definido y validado.
Flujo principal	**1.**El sistema registra la estación de origen. **2.**Registra la estación de destino. **3.**Registra los vehículos trasladados. **4.**Registra al operador responsable y la fecha del traslado. **5.**Actualiza la ubicación de los vehículos. **6.**Confirma el registro.

Flujos alternativos	**A1.** Si el origen, el destino o alguno de los vehículos no es válido, el sistema no confirma el traslado.

CU-23	Gestionar tarifas
Actor principal	Administrador.
Precondición	El administrador tiene permisos para gestionar las tarifas.
Flujo principal	**1.** El administrador accede a la gestión de tarifas. **2.** El sistema muestra las tarifas registradas por tipo de vehículo. **3.** El administrador registra o actualiza los valores de una nueva tarifa, indicando su fecha de vigencia. **4.** El sistema valida los datos. **5.** El sistema registra la nueva tarifa sin modificar los viajes ya cerrados. **6.** El sistema confirma la operación.
Flujos alternativos	**A1.** Si los valores son inválidos, el sistema solicita corregirlos. **A2.** Si la tarifa no tiene una fecha de vigencia válida, el sistema rechaza el registro.

CU-24	Gestionar promociones
Actor principal	Administrador.
Precondición	El administrador tiene permisos para gestionar las promociones.
Flujo principal	**1.** El administrador accede a la sección de promociones. **2.** El sistema muestra las promociones registradas. **3.** El administrador crea o modifica los datos de una promoción futura, incluyendo nombre, vigencia y condiciones de aplicación. **4.** El sistema valida los datos. **5.** El sistema registra la promoción. **6.** El sistema confirma la operación.
Flujos alternativos	**A1.** Si el período o las condiciones son inválidos, el sistema informa el error. **A2.** Las promociones vencidas no se aplican a nuevos viajes.

CU-25	Emitir reportes
Actor principal	Administrador.
Precondición	El administrador tiene acceso a los reportes del sistema.
Flujo principal	**1.** El administrador accede a la sección de reportes. **2.** El sistema ejecuta Seleccionar tipo de reporte (CU-26). **3.** El administrador selecciona el reporte que desea consultar. **4.** El sistema obtiene los datos necesarios. **5.** El sistema genera y presenta el reporte solicitado.
Flujos alternativos	**A1.** Si no hay datos para el período o criterio seleccionado, el sistema informa que no existen resultados. **A2.** Si ocurre un error al consultar los datos, el sistema informa que no pudo generar el reporte.

CU-26	Seleccionar tipo de reporte
Actor principal	Administrador.
Precondición	El administrador ingresó a la sección de reportes.
Flujo principal	**1.**El sistema presenta las opciones disponibles. **2.**El administrador selecciona una de las siguientes alternativas: –Viajes por usuario. –Uso por estación. –Vehículos con más fallas. **3.**El sistema solicita los filtros necesarios, si corresponde. **4.**El sistema devuelve los criterios seleccionados al caso de uso «Emitir reportes».
Flujos alternativos	**A1.**Si no se selecciona un tipo de reporte, el sistema solicita elegir una opción. **A2.**Si los filtros ingresados no son válidos, el sistema solicita corregirlos.

3. Diagrama de clases inicial

El diagrama inicial identifica las clases centrales del dominio, sus atributos y métodos con su visibilidad, las multiplicidades y las relaciones de asociación, agregación, composición y herencia. La jerarquía de herencia está formada por la clase abstracta Vehiculo y sus tres subclases.

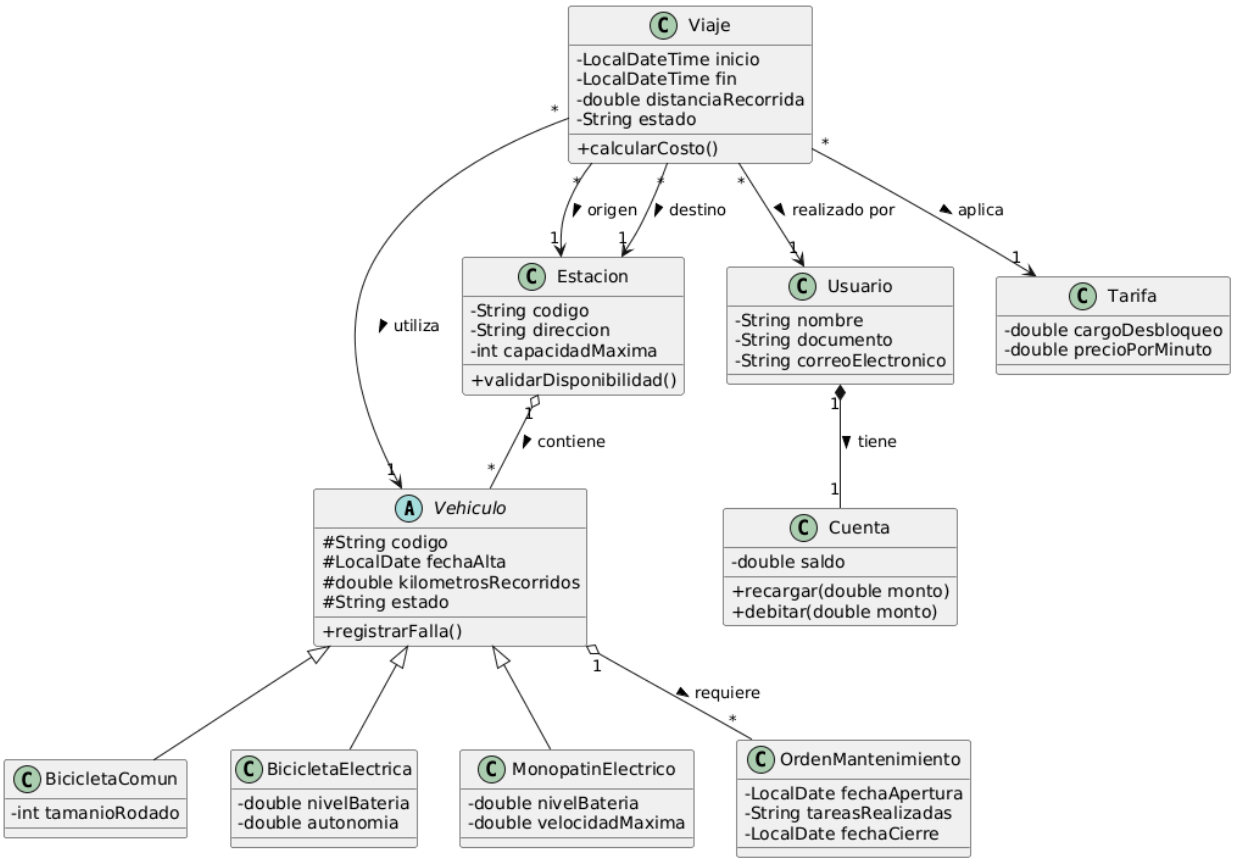


Figura 2. Diagrama de clases inicial.

3.1 Clases y atributos

Visibilidad: «+» público, «-» privado, «#» protegido. Los atributos de Vehiculo son protegidos para permitir el acceso directo desde sus subclases en la jerarquía.

Clase	Atributos	Métodos

Vehiculo
Abstracta
	`# codigo: String`
	`# fechaAlta: LocalDate`
	`# kilometrosRecorridos: double`
	`# estado: String`
		`+ registrarFalla(): void`

BicicletaComun *Hereda de Vehiculo*	`- tamanioRodado: int`	—
BicicletaElectrica *Hereda de Vehiculo*	`- nivelBateria: double` `- autonomia: double`	—
MonopatinElectrico *Hereda de Vehiculo*	`- nivelBateria: double` `- velocidadMaxima: double`	—

Estacion
Clase
	`- codigo: String`
	`- direccion: String`
	`- capacidadMaxima: int`
		`+ validarDisponibilidad(): boolean`

Clase	Atributos	Métodos

Usuario
Clase
	`- nombre: String`
	`- documento: String`
	`- correoElectronico: String`
		—

Cuenta *Clase*	`- saldo: double`	`+ recargar(monto: double): void` `+ debitar(monto: double): void`

Viaje
Clase
	`- inicio: LocalDateTime`
	`- fin: LocalDateTime`
	`- distanciaRecorrida: double`
	`- estado: String`
		`+ calcularCosto(): double`

Tarifa *Clase*	`- cargoDesbloqueo: double` `- precioPorMinuto: double`	—

OrdenMantenimiento
Clase
	`- fechaApertura: LocalDate`
	`- tareasRealizadas: String`
	`- fechaCierre: LocalDate`
		—

3.2 Relaciones y multiplicidades

Tipo	Clases	Multiplicidad	Descripción
Herencia	`BicicletaComun`, `BicicletaElectrica` y `MonopatinElectrico` → `Vehiculo`	**—**	Las tres subclases heredan de Vehiculo (generalización).
Agregación	`Estacion` ◇— `Vehiculo`	**1 a 0..***	Una estación alberga varios vehículos, pero la existencia del vehículo no depende estrictamente de la estación (puede estar en tránsito durante un viaje).
Composición	`Usuario` ◆— `Cuenta`	**1 a 1**	La existencia de la cuenta está acoplada al ciclo de vida del usuario registrado en el sistema.
Asociación	`Viaje` → `Estacion` (origen / destino)	**1 a ***	Cada viaje asocia 1 estación de origen y 1 estación de destino.
Asociación	`Viaje` → `Usuario` (realizado por)	**1 a ***	1 usuario puede realizar muchos viajes; cada viaje corresponde a 1 usuario.
Asociación	`Viaje` → `Vehiculo` (utiliza)	***** a 1**	1 viaje utiliza 1 vehículo en particular.
Asociación	`Viaje` → `Tarifa` (aplica)	***** a 1**	1 viaje aplica 1 regla de tarifa.
Asociación	`Vehiculo` — `OrdenMantenimiento` (requiere)	**1 a ***	1 vehículo puede requerir muchas órdenes de mantenimiento a lo largo de su vida útil.

4. Diagramas de secuencia

Se eligieron CU-05 Retirar vehículo (con flujos de error por reglas de negocio) y CU-08 Devolver vehículo (con flujo alternativo de estación llena). Los objetos usan nombres de clases del diagrama final (sección 6). Los objetos de dominio (u, v, e, c) ya fueron resueltos por los repositorios, paso omitido por claridad.

Diagrama	Caso de uso	Flujo alternativo o de error	Fragmentos utilizados
4.1	**CU-05 · Retirar vehículo**	Rechazos por reglas de negocio: usuario inhabilitado, viaje abierto, saldo insuficiente (RN01) y vehículo no disponible o con batería baja (RN03).	`alt` con cinco ramas (cuatro de error y el camino exitoso) y `opt` para la alerta de redistribución.
4.2	**CU-08 · Devolver vehículo**	Estación llena (RN07, RN08), con aviso de los 10 minutos sin cargo, y saldo negativo tras el débito (RN02).	`alt` (estación llena / hay lugar y saldo negativo), `loop` (puntos del recorrido y promociones vigentes) y `opt` (promoción aplicable).

4.1 CU-05 · Retirar vehículo

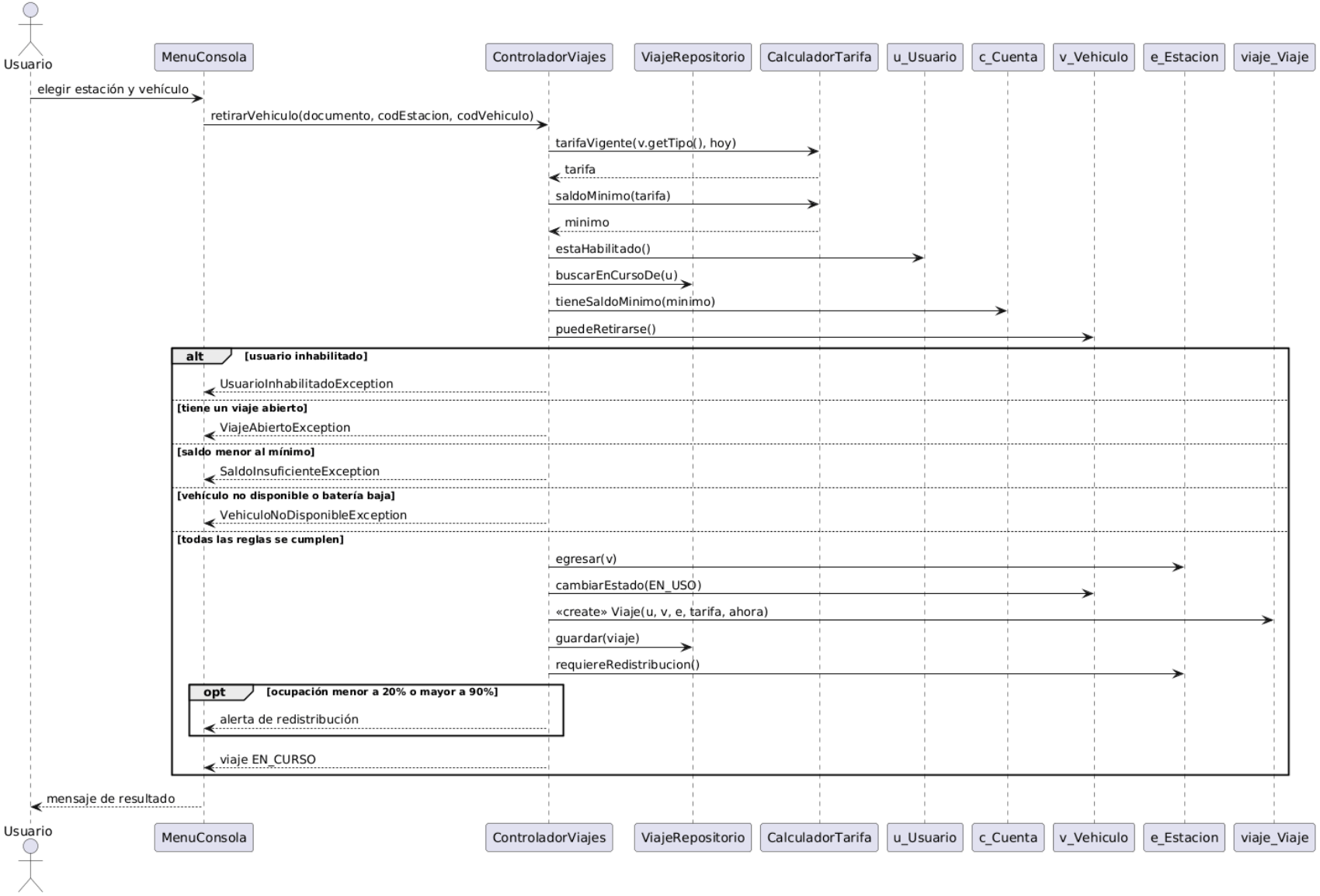


Figura 3. Diagrama de secuencia del caso de uso CU-05 · Retirar vehículo.

4.2 CU-08 · Devolver vehículo

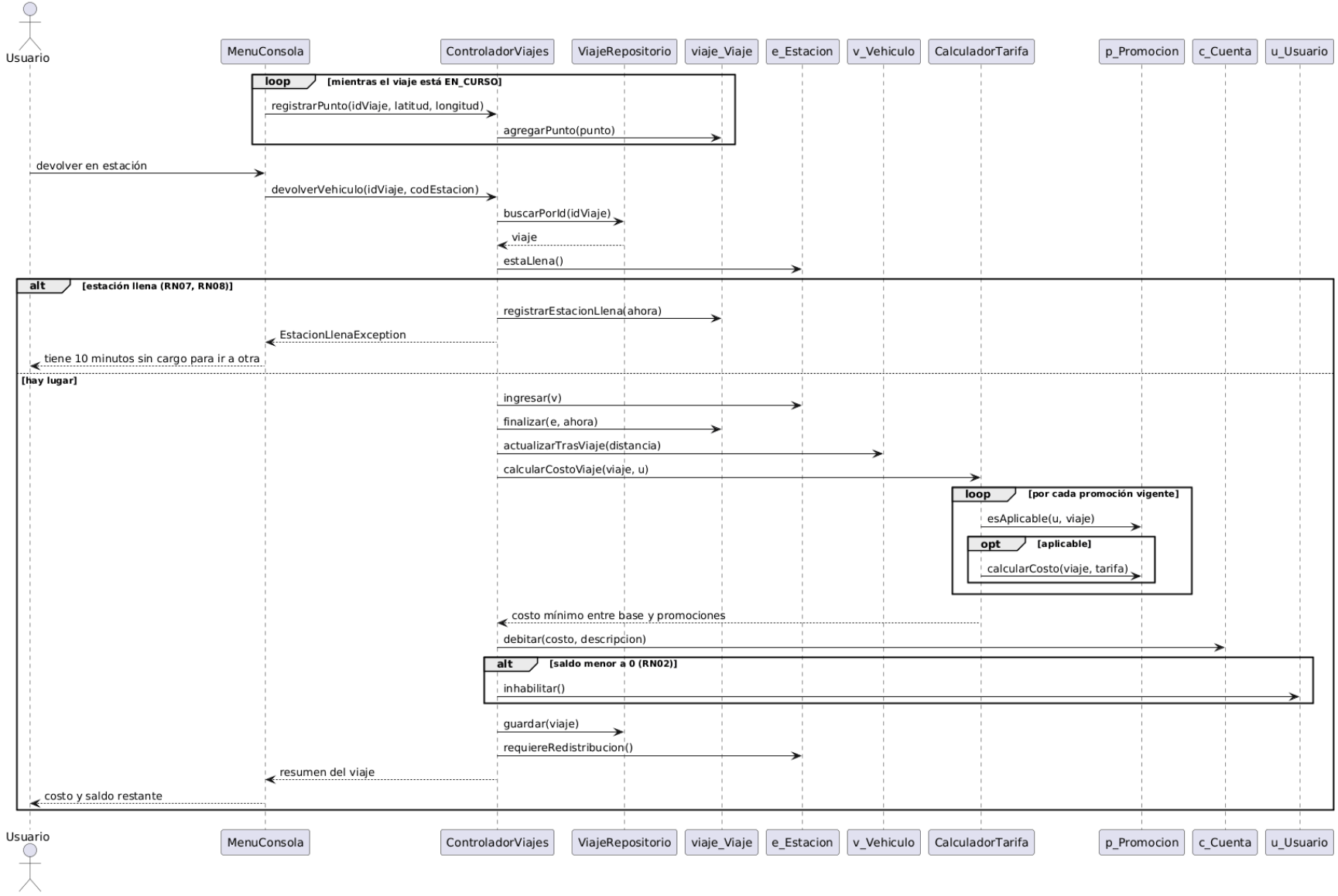


Figura 4. Diagrama de secuencia del caso de uso CU-08 · Devolver vehículo.

5. Aplicación de patrones de diseño

5.1. Principios SOLID

Se aplicarán tres principios SOLID para mejorar la organización de las responsabilidades, facilitar la incorporación de nuevas funcionalidades y reducir el acoplamiento entre las clases del sistema.

5.1.1. SRP — Principio de Responsabilidad Única

Problema detectado	En el sistema de movilidad compartida, una clase encargada de gestionar los viajes podría terminar concentrando demasiadas responsabilidades: validar el retiro de vehículos, calcular tarifas, evaluar promociones, actualizar el saldo de los usuarios y modificar el estado de los vehículos. Esto dificultaría el mantenimiento y las pruebas del sistema, ya que un cambio en una funcionalidad podría afectar a las demás.
Decisión de diseño	Separar las responsabilidades en clases especializadas. La clase `Viaje` se encargará de mantener la información del viaje y su estado; `ServicioTarifas` calculará el importe base; `ServicioPromociones` evaluará los beneficios disponibles; y `CuentaUsuario` administrará el saldo y sus movimientos. El `ControladorViajes` coordinará las operaciones necesarias para retirar y devolver un vehículo, delegando cada tarea a la clase correspondiente.
Clases involucradas	• `Viaje`: registra el inicio, la finalización, la duración, la distancia y el estado del viaje. • `ServicioTarifas`: calcula el precio según el tipo de vehículo, el tiempo de uso y la tarifa vigente. • `ServicioPromociones`: determina qué promoción corresponde aplicar y cuál es el importe más favorable. • `CuentaUsuario`: administra el saldo y registra los débitos y créditos. • `ControladorViajes`: coordina el flujo de los casos de uso relacionados con los viajes.
Ventaja obtenida	Cada clase tiene una responsabilidad claramente delimitada. Esto facilita la detección de errores, permite probar cada componente por separado y reduce el riesgo de introducir fallas al modificar una funcionalidad.

5.1.2. OCP — Principio Abierto/Cerrado

Problema detectado	El sistema contempla diferentes tipos de vehículos, tarifas y promociones. Si toda la lógica de cálculo se implementara mediante numerosas condiciones dentro de una única clase, agregar un nuevo tipo de vehículo o una nueva promoción exigiría modificar repetidamente el código existente.
Decisión de diseño	Diseñar los componentes de tarifas y promociones de manera que permitan incorporar nuevas variantes mediante nuevas implementaciones, sin modificar innecesariamente la lógica que coordina el cálculo del viaje. Se propone definir una abstracción común para las políticas de cálculo de tarifas y otra para las promociones. Las distintas implementaciones podrán resolver sus cálculos de acuerdo con sus reglas particulares. Por ejemplo, podrían existir distintas políticas de tarifa para bicicletas comunes, bicicletas eléctricas y monopatines eléctricos, así como promociones para el primer viaje y para el plan mensual.

Clases involucradas	• `PoliticaTarifa`: abstracción para el cálculo de tarifas. • `TarifaBicicletaComun`, `TarifaBicicletaElectrica` y `TarifaMonopatinElectrico`: implementaciones posibles de las políticas tarifarias. • `Promocion`: abstracción para representar una promoción. • `PromocionPrimerViaje` y `PromocionPlanMensual`: implementaciones posibles de los beneficios. • `ServicioTarifas` y `ServicioPromociones`: componentes que utilizan estas abstracciones.
Ventaja obtenida	Se facilita la incorporación de nuevas tarifas y promociones sin tener que reescribir el flujo principal de retiro y devolución de vehículos. Además, cada variante puede mantener sus propias reglas de cálculo.

5.1.3. DIP — Principio de Inversión de Dependencias

Problema detectado	Si el controlador de viajes dependiera directamente de implementaciones concretas de tarifas y promociones, quedaría fuertemente acoplado a ellas. Un cambio en el modo de calcular el precio o de evaluar los beneficios podría obligar a modificar el controlador. Una dependencia directa de mecanismos concretos de persistencia también dificultaría reemplazar la forma de almacenar la información.
Decisión de diseño	Hacer que los componentes de mayor nivel dependan de abstracciones en lugar de depender directamente de implementaciones concretas. Por ejemplo, `ControladorViajes` trabajará con abstracciones para calcular tarifas y evaluar promociones. De forma similar, una abstracción de repositorio permitirá acceder a los viajes y usuarios sin que el controlador deba conocer los detalles de lectura y escritura de los archivos CSV o JSON.
Clases involucradas	• `ControladorViajes`: coordina el proceso de retiro y devolución. • `PoliticaTarifa`: define el contrato para calcular tarifas. • `EvaluadorPromociones`: define el contrato para evaluar promociones. • `RepositorioViajes` y `RepositorioUsuarios`: abstracciones propuestas para acceder a los datos. • Implementaciones concretas de las políticas y repositorios.
Ventaja obtenida	Se reduce el acoplamiento entre la lógica del sistema y sus implementaciones concretas. Esto facilita las pruebas, permite sustituir componentes y favorece la evolución del sistema sin modificar innecesariamente las clases principales.

5.2. Patrones GRASP

Se aplicarán cuatro patrones GRASP para distribuir las responsabilidades entre los objetos de acuerdo con la información que poseen y las tareas que deben realizar.

5.2.1. Experto en Información (Information Expert)

Problema detectado	Para calcular el precio de un viaje se necesita conocer información como su fecha de inicio, fecha de finalización, duración y distancia. Si otra clase tuviera que mantener y calcular todos estos datos por su cuenta, se duplicaría la lógica y se aumentaría el acoplamiento.
Decisión de diseño	Asignar cada responsabilidad a la clase que dispone de la información necesaria para cumplirla. La clase `Viaje` será responsable de calcular su duración a partir de las fechas registradas y de obtener la distancia recorrida a partir de los puntos del recorrido. La clase `CuentaUsuario` será responsable de consultar y modificar el saldo de la cuenta. La clase `Estacion` será responsable de consultar su capacidad y ocupación. El cálculo de la tarifa final se delegará a los servicios especializados que utilizan los datos del viaje y las reglas tarifarias correspondientes.
Clases involucradas	• `Viaje`: fechas, recorrido, duración y estado. • `CuentaUsuario`: saldo y movimientos de cuenta. • `Estacion`: capacidad, vehículos alojados y ocupación. • `ServicioTarifas`: cálculo del precio base a partir de la información necesaria del viaje. • `ServicioPromociones`: evaluación de los beneficios aplicables.
Ventaja obtenida	Las responsabilidades quedan asignadas a las clases que poseen la información necesaria para cumplirlas. Se reduce la duplicación de cálculos y se mejora la cohesión del modelo.

5.2.2. Creador (Creator)

Problema detectado	La creación de objetos relacionados con un viaje, como los registros de movimientos o las órdenes de mantenimiento, podría quedar distribuida de manera desorganizada. Esto dificultaría mantener las relaciones entre los objetos y garantizar que se creen con los datos correctos.
Decisión de diseño	Asignar la creación de objetos a las clases que mantienen, contienen o utilizan estrechamente esos objetos. El `ControladorViajes`, al coordinar el retiro, solicitará la creación de un objeto `Viaje` con el usuario, el vehículo y la fecha de inicio correspondientes. La clase `CuentaUsuario`, al registrar un débito o crédito, generará el movimiento de cuenta asociado. De forma similar, el operador podrá solicitar la creación de una `OrdenMantenimiento` cuando corresponda gestionar una falla. La creación de una orden se realizará a partir de los datos del vehículo, el reporte de falla y el técnico asignado, según las reglas del sistema.
Clases involucradas	• `ControladorViajes`: coordina la creación del viaje. • `Viaje`: representa el viaje creado. • `CuentaUsuario`: registra los movimientos asociados a la cuenta. • `Operador`: solicita la generación de órdenes de mantenimiento. • `OrdenMantenimiento`: representa el trabajo de mantenimiento que debe realizarse.
Ventaja obtenida	La creación de los objetos se mantiene vinculada con las clases que conocen el contexto necesario para generarlos. Esto favorece la consistencia de las relaciones entre objetos y evita distribuir la construcción de entidades por toda la aplicación.

5.2.3. Controlador (Controller)

Problema detectado	Si la interfaz de consola contuviera directamente toda la lógica para retirar vehículos, calcular tarifas, evaluar promociones, registrar devoluciones y actualizar saldos, la presentación quedaría mezclada con las reglas de negocio. Esto dificultaría modificar los menús y probar las operaciones sin interactuar con la interfaz.
Decisión de diseño	Incorporar un `ControladorViajes` como intermediario entre la interfaz de consola y los objetos del dominio. Este controlador recibirá las solicitudes de los casos de uso, coordinará las validaciones y delegará las operaciones a las clases correspondientes. Por ejemplo, para retirar un vehículo, coordinará la validación del usuario, la comprobación del saldo mínimo, la verificación de disponibilidad y el inicio del viaje. Para devolverlo, coordinará el cálculo del importe, el débito, el registro de la finalización y la actualización del estado del vehículo.
Clases involucradas	• `InterfazConsola`: recibe las opciones y muestra los resultados. • `ControladorViajes`: coordina los casos de uso de retiro y devolución. • `CuentaUsuario`: valida y actualiza el saldo. • `Estacion` y `Vehiculo`: intervienen en la disponibilidad y devolución. • `ServicioTarifas` y `ServicioPromociones`: calculan el importe del viaje.
Ventaja obtenida	La interfaz se mantiene separada de la lógica de negocio. El controlador centraliza la coordinación de las operaciones sin asumir los cálculos y responsabilidades que corresponden a otras clases.

5.2.4. Polimorfismo (Polymorphism)

Problema detectado	El sistema administra vehículos de diferentes tipos, cuyas tarifas y características pueden variar. Resolver todas las diferencias mediante condicionales que comprueben el tipo de vehículo haría que la lógica se vuelva más extensa a medida que se agreguen nuevas variantes.
Decisión de diseño	Definir una jerarquía de vehículos con una clase abstracta `Vehiculo` y subclases concretas, como `BicicletaComun`, `BicicletaElectrica` y `MonopatinElectrico`. Las subclases podrán sobrescribir las operaciones que requieran comportamientos específicos. Por ejemplo, cada tipo podrá proporcionar su tarifa o aplicar el comportamiento correspondiente a sus características. Los vehículos eléctricos podrán gestionar su batería y pasar al estado `BATERIA_BAJA` cuando el nivel sea inferior al límite establecido. El sistema trabajará con referencias del tipo general `Vehiculo` y delegará en cada implementación el comportamiento que corresponda.
Clases involucradas	• `Vehiculo`: clase abstracta que define el comportamiento común. • `BicicletaComun`: representa una bicicleta sin batería. • `BicicletaElectrica`: incorpora el comportamiento relacionado con la batería. • `MonopatinElectrico`: incorpora el comportamiento relacionado con la batería. • `ServicioTarifas`: utiliza el tipo de vehículo para obtener el cálculo tarifario correspondiente.
Ventaja obtenida	Se aprovecha la herencia y la sobrescritura de métodos para representar las diferencias entre los tipos de vehículos. Esto reduce la necesidad de condicionales repetidos y permite agregar nuevos tipos mediante nuevas subclases, siempre que respeten el contrato definido por la clase abstracta.

5.3. Síntesis de las decisiones de diseño

Principio o patrón	Problema que resuelve	Decisión principal
SOLID — SRP	Acumulación de responsabilidades en una misma clase.	Separar viajes, cuentas, tarifas y promociones.
SOLID — OCP	Dificultad para agregar tarifas y promociones.	Incorporar nuevas implementaciones mediante abstracciones.
SOLID — DIP	Dependencia directa entre controladores e implementaciones concretas.	Depender de contratos o abstracciones.
GRASP — Experto	Asignación de tareas a clases que no poseen la información necesaria.	Dar cada responsabilidad a la clase que tiene los datos relevantes.
GRASP — Creador	Creación desorganizada de objetos relacionados.	Asignar la creación a las clases que mantienen o utilizan esos objetos.
GRASP — Controlador	Mezcla entre interfaz y lógica de negocio.	Centralizar la coordinación de los casos de uso en un controlador.
GRASP — Polimorfismo	Exceso de condicionales para diferenciar tipos de vehículos.	Utilizar herencia y sobrescritura de métodos.

5.4. Resultado esperado

La aplicación de estos principios y patrones busca obtener un diseño con responsabilidades claras, menor acoplamiento y mayor facilidad de mantenimiento. En particular, permite separar la coordinación de los viajes de los cálculos tarifarios, la evaluación de promociones, la administración de las cuentas y el comportamiento específico de cada tipo de vehículo.

Estas decisiones deberán reflejarse en el diagrama de clases final del punto 6 y mantenerse coherentes con los diagramas de casos de uso y de secuencia desarrollados en los puntos anteriores.

6. Diagrama de clases final

6. Diagrama de clases final

El diagrama de clases final se presenta en cuatro vistas complementarias para mantener la legibilidad. En conjunto forman un único modelo: dominio, tarifas y promociones, mantenimiento y operaciones, y controladores con persistencia.

Dominio: usuarios, cuentas, viajes, vehículos y estaciones

Jerarquía de herencia con sobreescritura: Vehiculo → BicicletaComun / VehiculoElectrico → BicicletaElectrica / MonopatinElectrico.



Figura 5.1. Dominio: usuarios, cuentas, viajes, vehículos y estaciones.

Tarifas y promociones

PoliticaTarifa y Promocion son abstracciones; se agregan nuevas tarifas o promociones con nuevas subclases, sin modificar los servicios (OCP, DIP, polimorfismo).

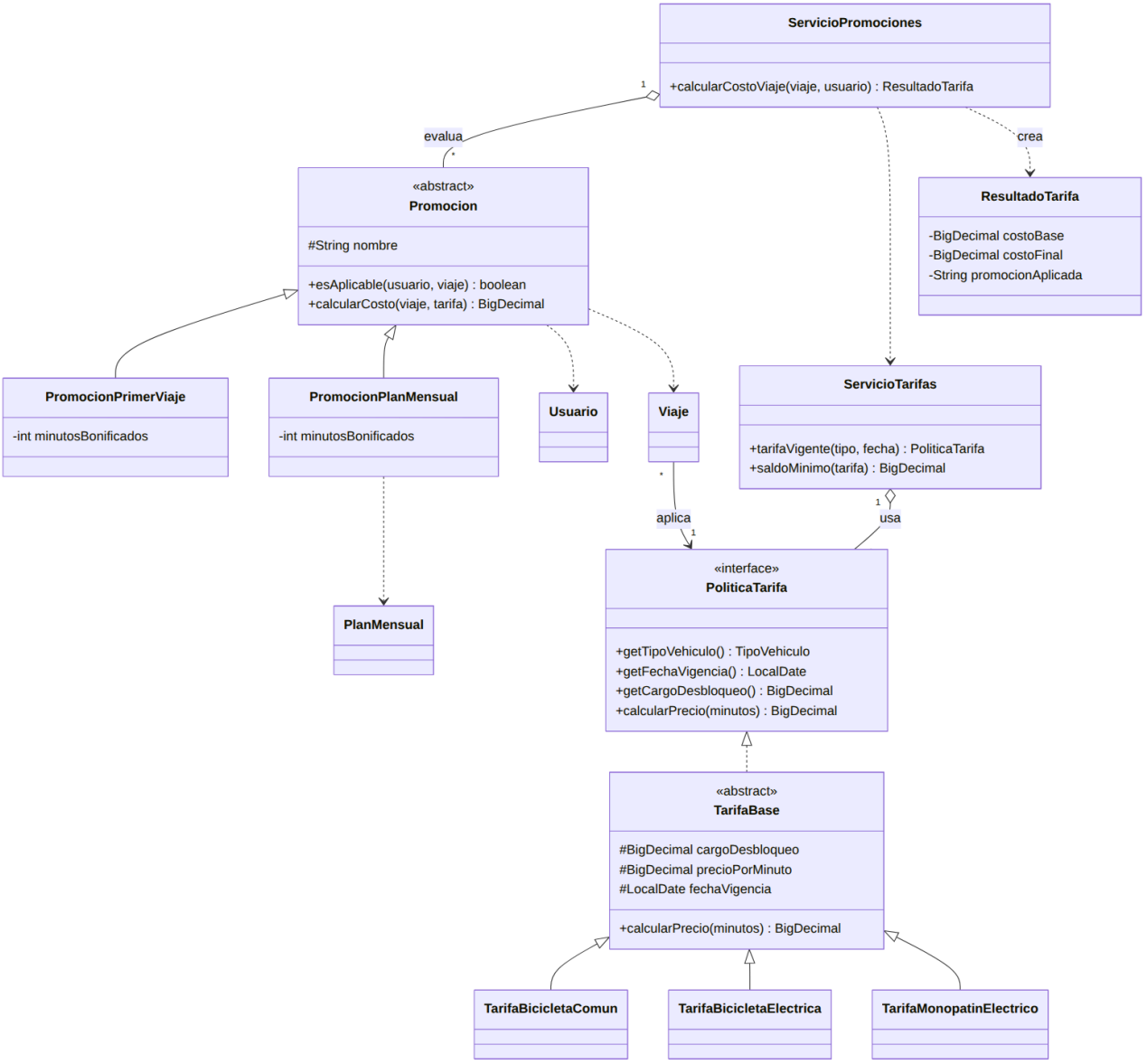


Figura 5.2. Tarifas y promociones.

Mantenimiento, redistribución, ajustes y empleados

Un vehículo tiene como máximo una orden de mantenimiento abierta. Empleado agrupa a Operador, Tecnico y Administrador.

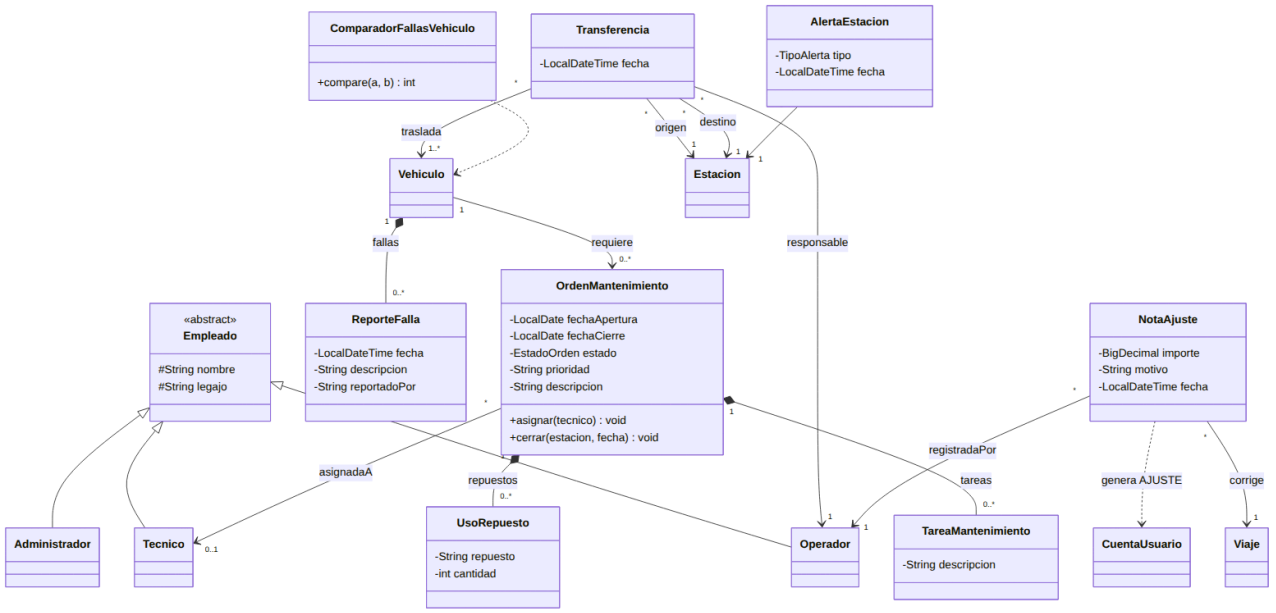


Figura 5.3. Mantenimiento, redistribución, ajustes y empleados.

Controladores, servicios, persistencia y soporte

La interfaz de consola solo conoce a los controladores, que dependen de interfaces de repositorio y del reloj inyectable (SRP, Controlador, DIP).

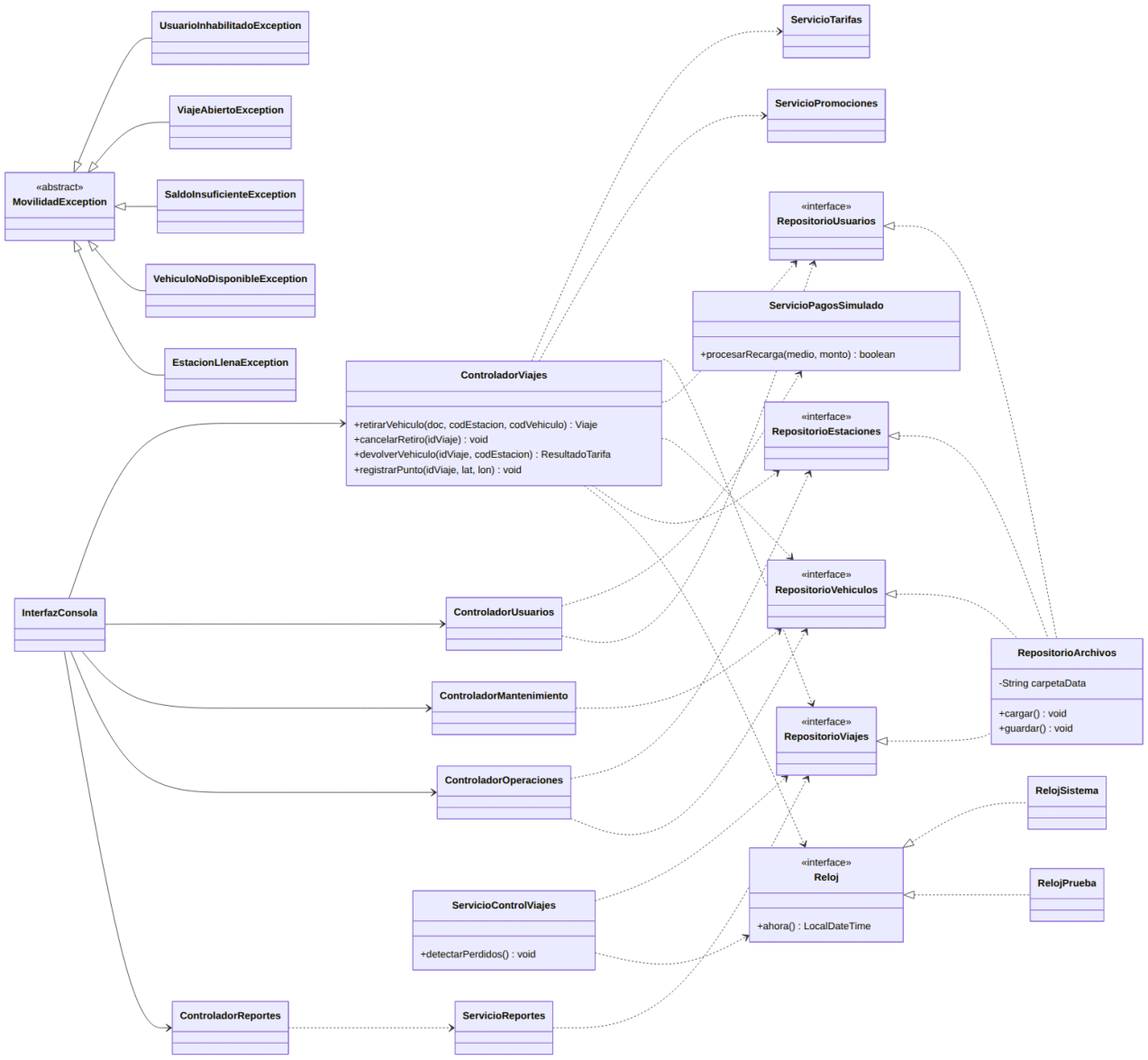


Figura 5.4. Controladores, servicios, persistencia y soporte.

6.1. Descripción general

El diagrama de clases final representa el diseño definitivo propuesto para el sistema municipal de movilidad compartida. Se rediseñó la estructura para separar las entidades del dominio, la coordinación de los casos de uso y los servicios encargados de realizar operaciones específicas.

El diseño se organiza en tres grupos principales:

Dominio	contiene las clases que representan los elementos del sistema y sus reglas de negocio, como `Usuario`, `CuentaUsuario`, `Viaje`, `Vehiculo`, `Estacion`, `PlanMensual`, `ReporteFalla`, `OrdenMantenimiento`, `Transferencia` y `NotaAjuste`.

Controladores y servicios	contiene las clases que coordinan los casos de uso y delegan las operaciones. Entre ellas se encuentran `ControladorViajes`, `ControladorUsuarios`, `ControladorMantenimiento`, `ControladorOperaciones`, `ControladorReportes`, `ServicioTarifas`, `ServicioPromociones`, `ServicioPagosSimulado` y `ServicioReportes`.
Persistencia y soporte	contiene las interfaces para acceder a los datos, su implementación mediante archivos, el reloj inyectable y las excepciones propias de las reglas de negocio.

Esta separación permite mantener las responsabilidades delimitadas y facilita la implementación posterior en Java.

6.2. Clases y responsabilidades principales

Clase o grupo de clases	Responsabilidad
`Usuario`, `CuentaUsuario` y `MovimientoCuenta`	Registrar usuarios, administrar saldos y conservar los movimientos de cuenta.
`MedioPago`, `Tarjeta` y `BilleteraVirtual`	Representar los medios de pago asociados a una cuenta.
`PlanMensual`	Registrar la vigencia, el precio y los viajes consumidos de una suscripción.
`Vehiculo` y sus subclases	Representar los tipos de vehículos y sus diferencias de comportamiento.
`Estacion`	Administrar la capacidad, la ocupación y los vehículos ubicados en cada estación.
`Viaje` y `PuntoRecorrido`	Registrar el inicio, la finalización, la distancia, la duración y el recorrido.
`PoliticaTarifa` y sus implementaciones	Definir las tarifas correspondientes a cada tipo de vehículo y su fecha de vigencia.
`Promocion` y sus subclases	Representar los beneficios disponibles y calcular sus importes.
`ServicioTarifas` y `ServicioPromociones`	Calcular el precio base y seleccionar el beneficio más favorable.
`ReporteFalla` y `OrdenMantenimiento`	Registrar fallas y gestionar las tareas de mantenimiento de los vehículos.
`Transferencia` y `AlertaEstacion`	Representar la redistribución de vehículos y las alertas de ocupación.
`NotaAjuste`	Registrar correcciones de saldo asociadas a un viaje sin modificar el viaje original.
`ServicioControlViajes`	Detectar viajes que superaron el plazo para devolver el vehículo.
`ServicioReportes`	Generar reportes de viajes por usuario, uso por estación y vehículos con más fallas.
`RepositorioArchivos`	Persistir los datos en archivos CSV o JSON dentro de la carpeta `data`.
`Reloj`, `RelojSistema` y `RelojPrueba`	Obtener fechas y horas, permitiendo controlar el tiempo durante las pruebas.

6.3. Relaciones principales del diagrama

Usuarios, cuentas y viajes

Cada usuario posee una cuenta, y la cuenta registra cero o muchos movimientos. Un usuario puede realizar varios viajes a lo largo del tiempo, aunque las reglas de negocio impiden que inicie uno nuevo si ya tiene otro abierto.

Cada viaje corresponde a un único vehículo y tiene una estación de retiro. La estación de devolución puede ser distinta y se registra cuando la devolución se completa. El viaje conserva los puntos de recorrido y los datos necesarios para determinar su duración, distancia e importe.

Vehículos y estaciones

Se define una clase abstracta `Vehiculo`, de la cual heredan `BicicletaComun` y `VehiculoElectrico`. Esta última es la clase general de `BicicletaElectrica` y `MonopatinElectrico`.

La herencia permite representar las características comunes en `Vehiculo` y concentrar en `VehiculoElectrico` el comportamiento relacionado con la batería. Las bicicletas comunes no necesitan administrar un nivel de batería.

Un vehículo puede estar ubicado en una estación o encontrarse en uso. Las asociaciones con los viajes y las estaciones permiten representar su historial y su ubicación actual.

Tarifas y promociones

La interfaz `PoliticaTarifa` define las operaciones comunes para consultar el tipo de vehículo, la fecha de vigencia y calcular el precio. Las clases de tarifa concretas implementan esa política para cada tipo de vehículo.

La clase abstracta `Promocion` permite representar los beneficios de manera extensible. `PromocionPrimerViaje` y `PromocionPlanMensual` implementan las reglas particulares de cada beneficio.

`ServicioTarifas` selecciona la tarifa vigente al momento de inicio del viaje. `ServicioPromociones` compara el precio normal con los importes de las promociones aplicables y selecciona el menor. El resultado se representa mediante `ResultadoTarifa`.

Mantenimiento y redistribución

Cada vehículo puede tener varios reportes de falla y órdenes de mantenimiento a lo largo del tiempo, pero solo puede mantener una orden abierta simultáneamente. Una orden registra sus tareas, los repuestos utilizados, el técnico asignado y las fechas correspondientes.

La clase `Transferencia` registra el origen, el destino, el operador responsable y los vehículos trasladados. `AlertaEstacion` representa las situaciones en las que una estación tiene una ocupación inferior al 20 % o superior al 90 %.

Persistencia y soporte

Los controladores y servicios dependen de interfaces de repositorio, en lugar de depender directamente de los detalles de los archivos. `RepositorioArchivos` implementa esas interfaces y permite conservar los datos entre ejecuciones.

La interfaz `Reloj` permite sustituir el reloj real por `RelojPrueba`, facilitando las pruebas de límites temporales. Las excepciones propias permiten representar errores de negocio, como saldo insuficiente, usuario deshabilitado, vehículo no disponible o estación saturada.

6.4. Nota de cambios respecto del diagrama de clases inicial

El diseño final incorpora los siguientes cambios para reflejar las decisiones adoptadas en el punto 5:

Separación de responsabilidades — SRP y GRASP Controlador	Se distingue la coordinación de los casos de uso, realizada por los controladores, de los cálculos tarifarios, la evaluación de promociones, las recargas, el mantenimiento y la generación de reportes. Esto evita concentrar toda la lógica del sistema en una sola clase.
Incorporación de abstracciones — OCP y DIP	Se incorporan las interfaces `PoliticaTarifa`, `MedioPago` y las interfaces de repositorio. De esta manera, las funcionalidades pueden extenderse o cambiar de implementación con menor impacto sobre los controladores.

Uso de herencia y polimorfismo	Se organiza la jerarquía `Vehiculo` y `VehiculoElectrico`, junto con sus subclases concretas. También se define la jerarquía de `Empleado`, integrada por `Operador`, `Tecnico` y `Administrador`. Estas relaciones permiten representar comportamientos comunes y especializados.
Especialización del cálculo económico	Se separan las tarifas, las promociones, el resultado del cálculo y los movimientos de cuenta. Esto permite registrar el importe aplicado al viaje, conservar la promoción utilizada y mantener la trazabilidad de los débitos, recargas y ajustes.
Representación explícita del mantenimiento y la redistribución	Se incorporan clases específicas para reportes de falla, órdenes, tareas, uso de repuestos, transferencias y alertas de estación. Estas clases permiten registrar el historial y aplicar las reglas de negocio correspondientes.
Persistencia y control temporal	Se agregan repositorios abstractos, una implementación para archivos y un reloj inyectable. Estas incorporaciones facilitan cumplir los requisitos técnicos de almacenamiento y probar las reglas que dependen del tiempo.
Ordenamiento y excepciones	`Viaje` implementa el criterio de orden natural por fecha, mientras que `ComparadorFallasVehiculo` permite ordenar los vehículos por cantidad de fallas. Además, se agregan excepciones específicas para los errores de negocio.

6.5. Reglas de negocio reflejadas en el diseño final

El modelo contempla las siguientes reglas relevantes para la implementación:

- El saldo mínimo para retirar un vehículo se calcula a partir del cargo de desbloqueo y 30 minutos de uso, según la tarifa aplicable.
- La duración del viaje se redondea hacia arriba. La tarifa vigente se determina según el tipo de vehículo y la fecha de inicio, y el viaje conserva el importe calculado.
- Las promociones se evalúan por separado y se selecciona el beneficio más favorable. El plan mensual dura 30 días calendario y permite hasta 20 viajes.
- Un débito puede dejar el saldo negativo. En ese caso, el usuario queda inhabilitado para iniciar nuevos viajes hasta regularizarlo.
- La devolución, la cancelación, la saturación de estaciones y la detección de vehículos perdidos respetan sus límites temporales.
- Los vehículos eléctricos controlan su batería; el mantenimiento y las transferencias permiten restaurarla según las reglas definidas.
- Los cambios de estado se persisten en archivos y los importes monetarios se representan con `BigDecimal`, redondeados a dos decimales.

En conclusión, el diagrama de clases final transforma el modelo conceptual en una estructura orientada a objetos con responsabilidades delimitadas, herencia, polimorfismo, abstracciones y componentes de soporte. El diseño servirá como referencia para la codificación en Java y deberá mantenerse coherente con los diagramas de casos de uso y de secuencia de esta etapa.